

Лабораторная работа №6

Решение моделей в непрерывном и дискретном времени

Александр Эдуардович Аскеров

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Модель экспоненциального роста	8
3.2	Система Лоренца	11
3.3	Модель Лотки–Вольтерры	14
3.4	Задания для самостоятельного выполнения	17
4	Выводы	37
	Список литературы	38

Список иллюстраций

3.1	Модель экспоненциального роста 1	8
3.2	Модель экспоненциального роста 2	9
3.3	Модель экспоненциального роста 3	9
3.4	Модель экспоненциального роста 4	10
3.5	Модель экспоненциального роста 5	10
3.6	Система Лоренца 1	11
3.7	Система Лоренца 2	12
3.8	Система Лоренца 3	13
3.9	Модель Лотки–Вольтерры 1	14
3.10	Модель Лотки–Вольтерры 2	15
3.11	Модель Лотки–Вольтерры 3	16
3.12	Модель Лотки–Вольтерры 4	16
3.13	Решение	17
3.14	График	18
3.15	Анимация	19
3.16	Решение	20
3.17	График	21
3.18	Анимация	22
3.19	Решение	23
3.20	График	24
3.21	Анимация	25
3.22	Решение	26
3.23	График	27
3.24	График	28
3.25	Фазовый портрет	29
3.26	Решение	30
3.27	График	30
3.28	Фазовый портрет	31
3.29	Анимация	31
3.30	Решение	32
3.31	График	33
3.32	Фазовый портрет	33
3.33	Анимация	34
3.34	Решение	35
3.35	График	35
3.36	Фазовый портрет	36

3.37	Анимация	36
------	--------------------	----

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 6.2.
2. Выполните задания для самостоятельной работы.

3 Выполнение лабораторной работы

Повторим примеры [1].

3.1 Модель экспоненциального роста

```
1 # подключаем необходимые пакеты
2 import Pkg
3 Pkg.add("DifferentialEquations")
4 using DifferentialEquations

22685.1 ms ✓ OrdinaryDiffEqBDF
35888.6 ms ✓ OrdinaryDiffEqFIRK
53102.6 ms ✓ OrdinaryDiffEqDefault
7473.7 ms ✓ OrdinaryDiffEq
7756.7 ms ✓ DelayDiffEq
153319.3 ms ✓ BoundaryValueDiffEqMIRK
219438.2 ms ✓ BoundaryValueDiffEqFIRK
8725.5 ms ✓ BoundaryValueDiffEq
11858.5 ms ✓ DifferentialEquations
260 dependencies successfully precompiled in 500 seconds. 222 already
precompiled.
2 dependencies had output during precompilation:
[ MKL_jll
  Downloading artifact: IntelOpenMP
  Downloading artifact: oneTBB
]
[ LinearSolve
  Downloading artifact: MKL
]
```

Рисунок 3.1: Модель экспоненциального роста 1


```

1 # задаём описание модели с начальными условиями
2 a = 0.98
3 f(u,p,t) = a*u
4 u0 = 1.0
5 # задаём интервал времени
6 tspan = (0.0, 1.0)
7 # решение
8 prob = ODEProblem(f,u0,tspan)
9 sol = solve(prob)

retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.10042494449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
 1.1034222047865465
 1.4121908713484919
 1.9730384457359198
 2.6644561424814266

```

Рисунок 3.2: Модель экспоненциального роста 2

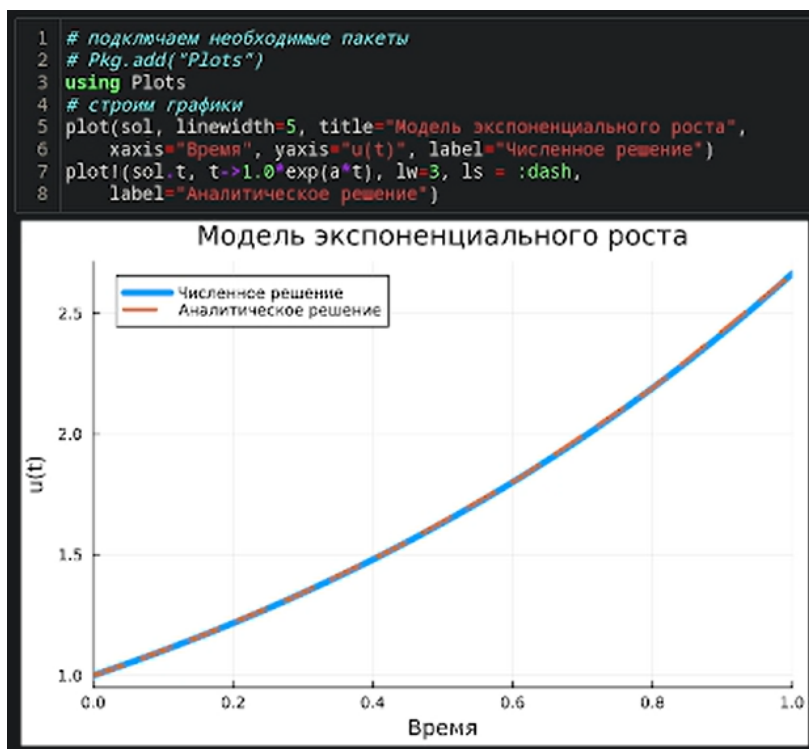


Рисунок 3.3: Модель экспоненциального роста 3

```

1 # задаём точность решения
2 sol = solve(prob, abstol=1e-8, reltol=1e-8)
3 println(sol)

```

Рисунок 3.4: Модель экспоненциального роста 4

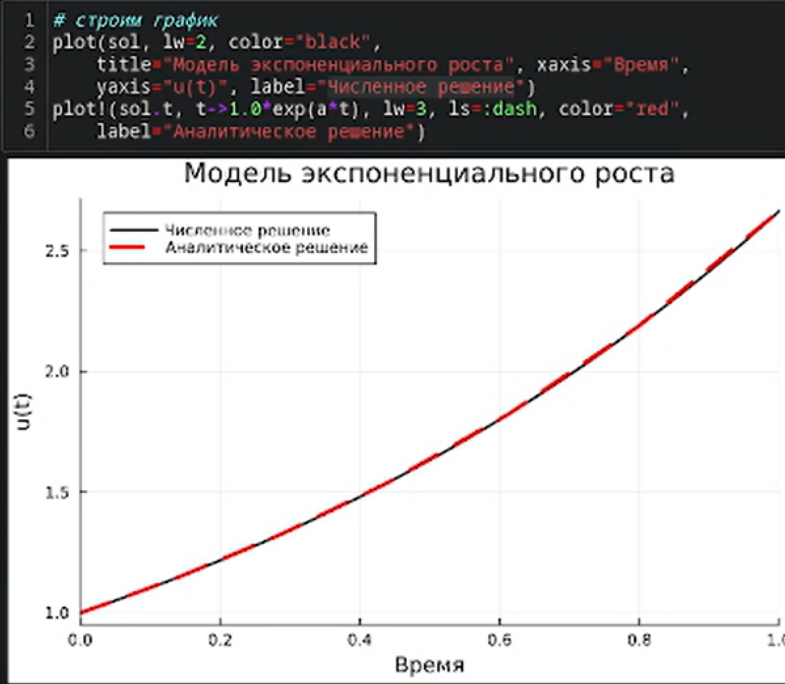


Рисунок 3.5: Модель экспоненциального роста 5

3.2 Система Лоренца

```
1 # подключаем необходимые пакеты
2 # import Pkg
3 # Pkg.add("DifferentialEquations")
4 using DifferentialEquations, Plots;
5 # задаём описание модели
6 function lorenz!(du,u,p,t)
7     sigma, rho, beta = p
8     du[1] = sigma*(u[2]-u[1])
9     du[2] = u[1]*(rho-u[3]) - u[2]
10    du[3] = u[1]*u[2] - beta*u[3]
11 end
12 # задаём начальное условие
13 u0 = [1.0, 0.0, 0.0]
14 # задаём значения параметров
15 p = (10, 28, 8/3)
16 # задаём интервал времени
17 tspan = (0.0, 100.0)
18 # решение
19 prob = ODEProblem(lorenz!,u0,tspan,p)
20 sol = solve(prob)

retcode: Success
Interpolation: 3rd order Hermite
t: 1292-element Vector{Float64}:
 0.0
 3.5678604836301404e-5
 0.0003924646531993154
 0.003262408731175873
 0.009058076622686189
```

Рисунок 3.6: Система Лоренца 1

```

1 # подключаем необходимые пакеты
2 # Pkg.add("Plots")
3 using Plots
4 # строим график
5 plot(sol, vars=(1,2,3), lw=2, title="Аттрактор Лоренца",
6      xaxis="x", yaxis="y", zaxis="z", legend=false)

```

Warning: To maintain consistency with solution indexing, keyword argument vars will be removed in a future version. Please use keyword argument idxs instead.

caller = ip:0x0

@ Core :-1

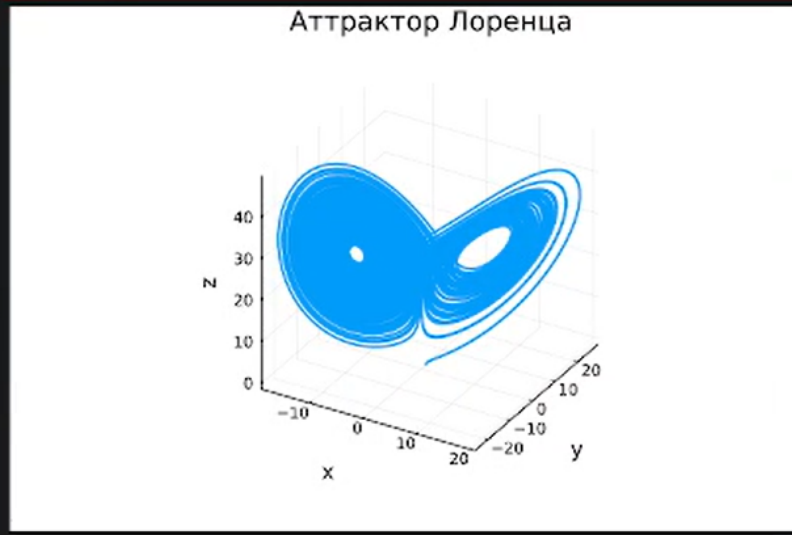


Рисунок 3.7: Система Лоренца 2

```

1 # отключаем интерполяцию
2 plot(sol, vars=(1,2,3), denseplot=false, lw=1,
3     title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z",
4     legend=false)

```

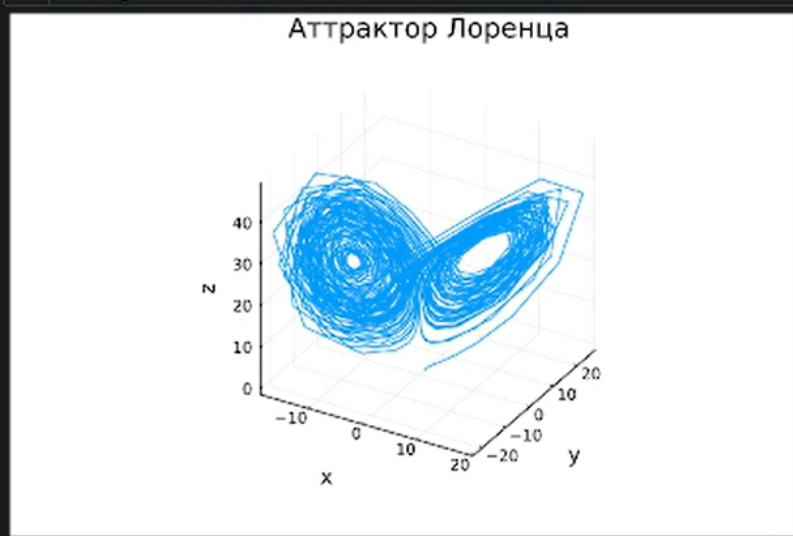


Рисунок 3.8: Система Лоренца 3

3.3 Модель Лотки–Вольтерры

```
1 import Pkg
2 Pkg.add("ParameterizedFunctions")

Resolving package versions...
Installed Tricks ————— v0.1.13
Installed FindFirstFunctions — v1.4.2
Installed Bijections ————— v0.2.2
Installed Unitful ————— v1.25.1
Installed OffsetArrays ————— v1.17.0
Installed CompositeTypes ————— v0.1.4
Installed MutableArithmetics — v1.6.7
Installed DomainSets ————— v0.7.16
Installed JuliaFormatter ————— v2.2.0
Installed MultivariatePolynomials — v0.5.13
Installed JuliaSyntax ————— v0.4.10
Installed Symbolics ————— v6.57.0
Installed CommonMark ————— v0.9.1
Installed DispatchDoctor ————— v0.4.26
Installed Glob ————— v1.3.1
Installed MLStyle ————— v0.4.17
Installed ImplicitDiscreteSolve — v1.2.0
Installed DynamicPolynomials — v0.6.4
Installed SymbolicSolve ————— v0.2.0
```

Рисунок 3.9: Модель Лотки–Вольтерры 1

```

1 using ParameterizedFunctions, DifferentialEquations, Plots;
2 # задаём описание модели
3 lv! = @ode_def LotkaVolterra begin
4     dx = a*x - b*x*y
5     dy = -c*y + d*x*y
6 end a b c d
7
8 # задаём начальное условие
9 u0 = [1.0, 1.0]
10 # задаём значения параметров
11 p = (1.5, 1.0, 3.0, 1.0)
12 # задаём интервал времени
13 tspan = (0.0, 10.0)
14 # решение
15 prob = ODEProblem(lv!, u0, tspan, p)
16 sol = solve(prob)

Warning: Independent variable t should be defined with @independent_v
variables t.
@ ModelingToolkit ~/.julia/packages/ModelingToolkit/8TOXs/src/utils.j
l:121

retcode: Success
Interpolation: 3rd order Hermite
t: 34-element Vector{Float64}:
 0.0
 0.0776084743154256
 0.2326451370670694
 0.42911851563726466
 0.679082199936808
 0.9444046279774128
 1.2674601918628516
 1.61929140093895

```

Рисунок 3.10: Модель Лотки–Вольтерры 2

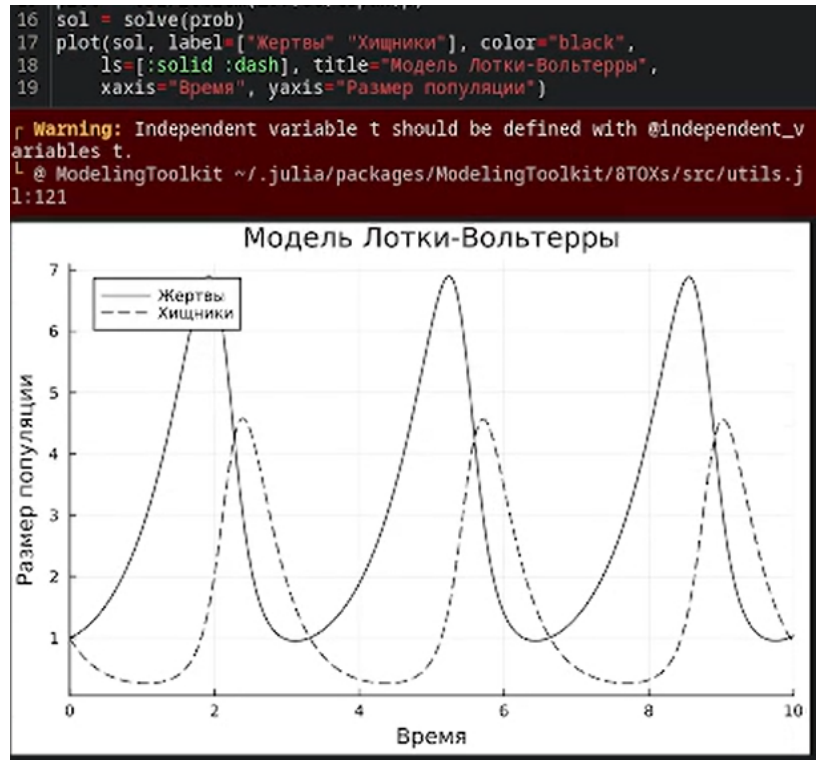


Рисунок 3.11: Модель Лотки-Вольтерры 3

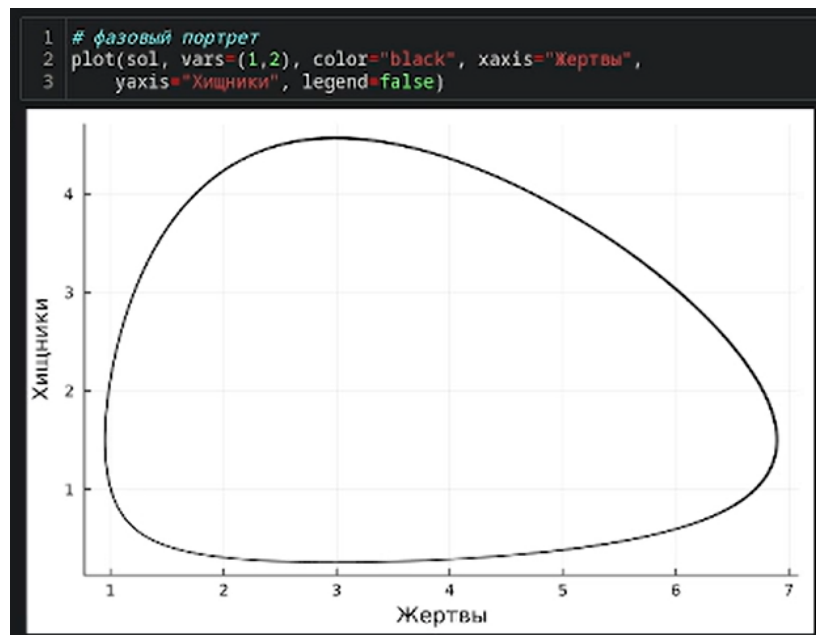


Рисунок 3.12: Модель Лотки-Вольтерры 4

3.4 Задания для самостоятельного выполнения

№1.

Реализуем и проанализируем модель роста численности изолированной популяции (модель Мальтуса):

$$\dot{x} = ax, \quad a = b - c,$$

где $x(t)$ – численность изолированной популяции в момент времени t , a – коэффициент роста популяции, b – коэффициент рождаемости, c – коэффициент смертности. Построим соответствующие графики (в том числе с анимацией).

```
1 # Задание 1
2 f(u,p,t) = a*u
3 b = 0.03
4 c = 0.01
5 a = b - c
6 tspan = (0.0, 100.0)
7 u0 = 1
8 prob = ODEProblem(f,u0,tspan)
9 sol = solve(prob)

retcode: Success
Interpolation: 3rd order Hermite
t: 9-element Vector{Float64}:
 0.0
 0.21871613178830912
 2.4058774496714004
11.977629518090481
27.310182664671096
45.36541085662215
67.90141413844084
93.90601659343899
100.0
u: 9-element Vector{Float64}:
 1.0
 1.0043839039505202
 1.0492939915280628
 1.270680508644512
 1.7266854419740234
 2.4776441176889845
 3.888517373873031
 6.541190667266496
 7.389047795846145
```

Рисунок 3.13: Решение

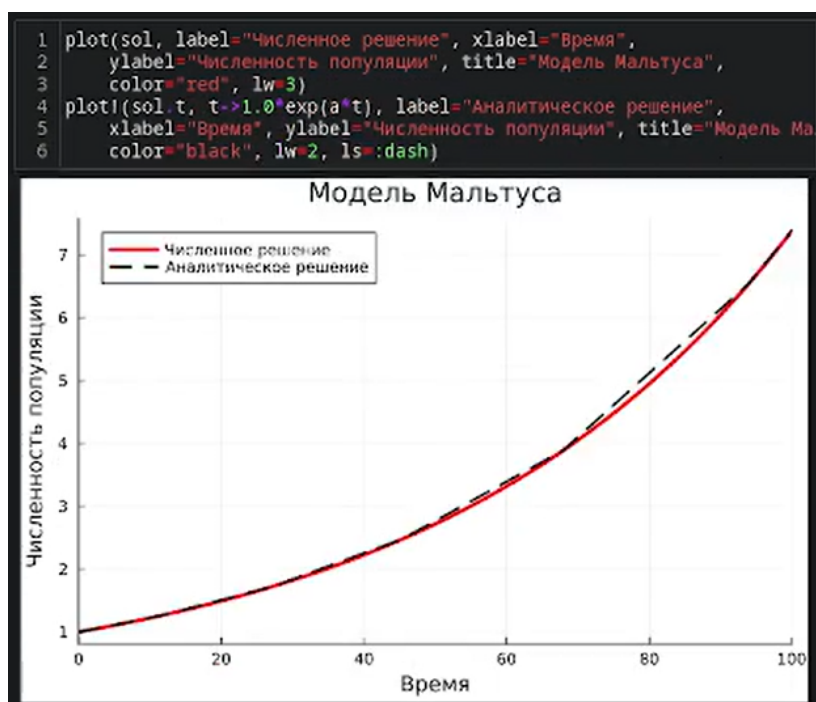


Рисунок 3.14: График

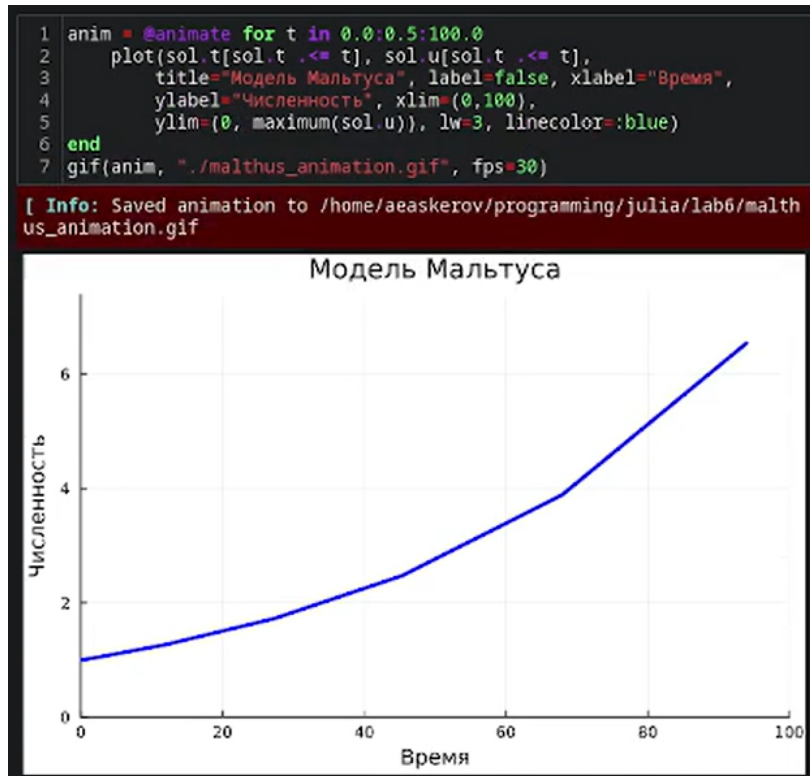


Рисунок 3.15: Анимация

№2.

Реализуем и проанализируем логистическую модель роста популяции:

$$\dot{x} = rx\left(1 - \frac{x}{k}\right), \quad r > 0, \quad k > 0,$$

где r – коэффициент роста популяции, k – потенциальная ёмкость экологической системы (предельное значение численности популяции). Построим соответствующие графики (в том числе с анимацией).

```
1 # Задание 2
2 f(u,p,t) = r*u*(1-u/k)
3 r = 0.7
4 k = 10000
5 u0 = 10
6 tspan = (0.0, 40.0)
7 prob = ODEProblem(f,u0,tspan)
8 sol = solve(prob)

retcode: Success
Interpolation: 3rd order Hermite
t: 24-element Vector{Float64}:
 0.0
 0.10741773231955752
 0.4313548314582653
 0.8957889319312652
 1.450120467893338
 2.126380894181169
```

Рисунок 3.16: Решение

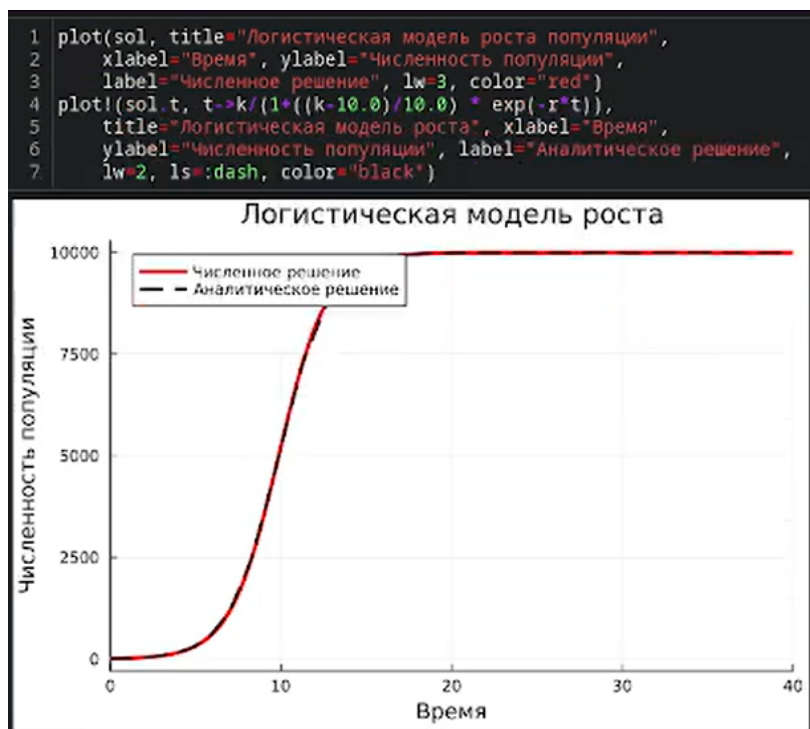


Рисунок 3.17: График

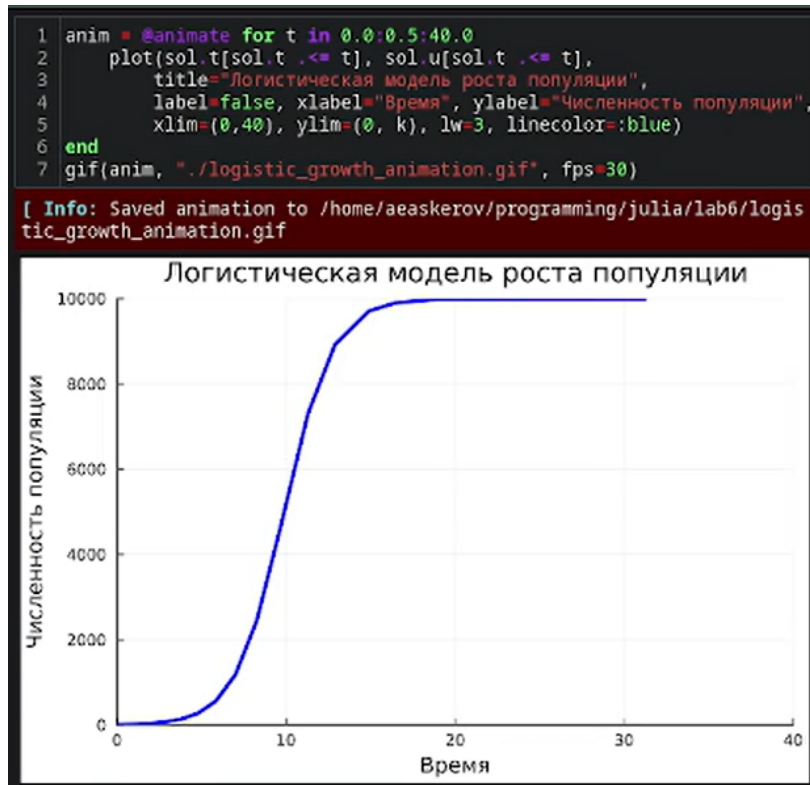


Рисунок 3.18: Анимация

№3.

Реализуем и проанализируем логистическую модель эпидемии Кермака–Маккендрика (SIR-модель):

$$\begin{cases} \dot{s} = -\beta i s, \\ \dot{i} = \beta i s - \nu i, \\ \dot{r} = \nu i, \end{cases}$$

где $s(t)$ – численность восприимчивых к болезни индивидов в момент времени t , $i(t)$ – численность инфицированных индивидов в момент времени t , $r(t)$ – численность переболевших индивидов в момент времени t , N – это сумма этих трёх, β – коэффициент интенсивности контактов индивидов с последующим инфицированием, ν – коэффициент интенсивности выздоровления соответственно

инфицированных индивидов. Численность популяции считается постоянной, т.е. $\dot{s} + \dot{i} + \dot{r} = 0$. Построим соответствующие графики (в том числе с анимацией).

```

1 # Задание 3
2 function SIR(u,p,t)
3     (S,I,R) = u
4     (β, v) = p
5     N = S + I + R
6     dS = -(β*I*S)/N
7     dI = (β*I*S)/N - v*I
8     dR = v*I
9     return [dS, dI, dR]
10 end
11 δt = 0.1
12 tmax = 100.0
13 tspan = (0.0, tmax)
14 u0 = [555.0, 10.0, 0.0] # S, I, R
15 p = [0.2, 0.1] # β, v
16 prob_ode = ODEProblem(SIR, u0, tspan, p)
17 sol_ode = solve(prob_ode, dt=δt)

retcode: Success
Interpolation: 3rd order Hermite
t: 16-element Vector{Float64}:
 0.0
 0.1
 0.8508492944639893
 2.7247850099533113
 5.683545539112153
 9.527301085497648
14.651970878295813
21.043273592743866

```

Рисунок 3.19: Решение

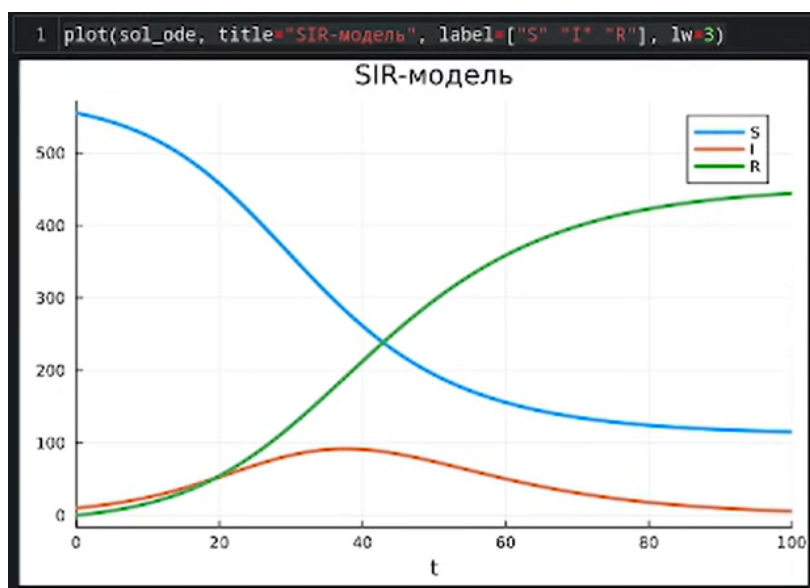


Рисунок 3.20: График

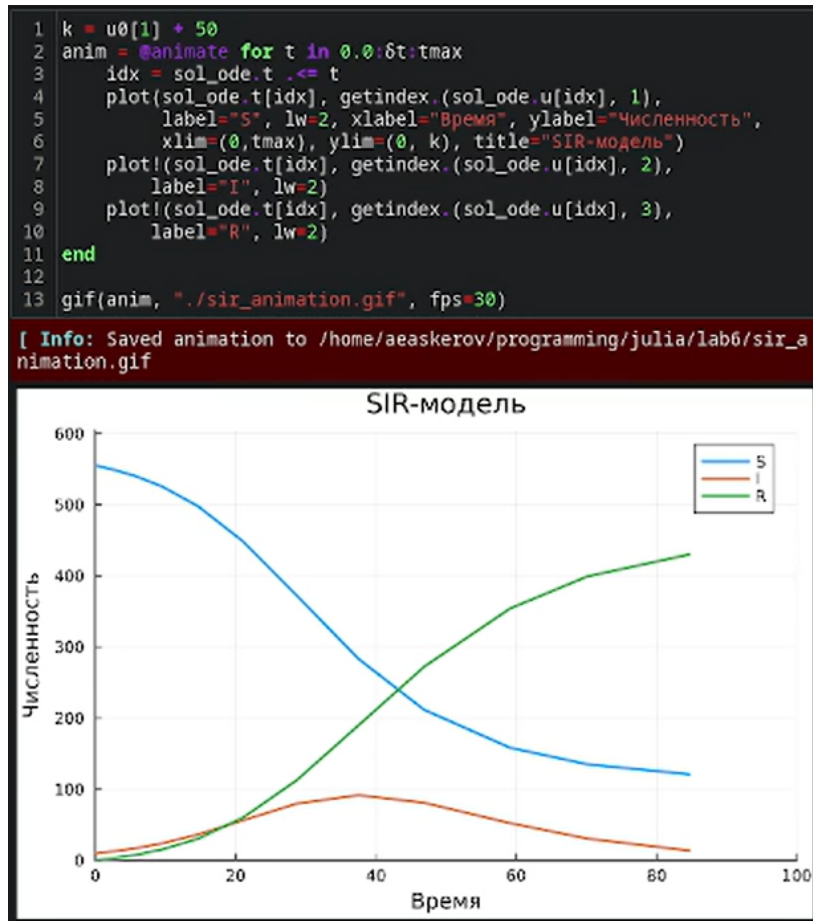


Рисунок 3.21: Анимация

№4.

Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed).

$$\begin{cases} \dot{s}(t) = -\frac{\beta}{N}s(t)i(t), \\ \dot{e}(t) = \frac{\beta}{N}s(t)i(t) - \delta e(t), \\ \dot{i}(t) = \delta e(t) - \gamma i(t), \\ \dot{r}(t) = \gamma i(t), \end{cases}$$

Размер популяции сохраняется:

$$s(t) + e(t) + i(t) + r(t) = N$$

Исследуем и сравним с SIR.

```

1  # Задание 4
2  function SEIR(u,p,t)
3      (S,E,I,R) = u
4      (β, γ, δ) = p
5      N = S + E + I + R
6      dS = -(β*I*S)/N
7      dE = (β*S*I)/N - δ*E
8      dI = δ*E - γ*I
9      dR = γ*I
10     return [dS, dE, dI, dR]
11 end
12 δt = 0.1
13 tmax = 100.0
14 tspan = (0.0, tmax)
15 u0 = [555.0, 10.0, 10.0, 0.0]
16 p = [0.2, 0.1, 0.3]
17 prob_ode = ODEProblem(SEIR, u0, tspan, p)
18 sol_ode = solve(prob_ode, dt=δt)

retcode: Success
Interpolation: 3rd order Hermite
t: 26-element Vector{Float64}:
 0.0
 0.1
 0.4606358685522852
 1.1147568458380954
 1.9938878625644165
 3.1234377177125676
 4.525708059287021

```

Рисунок 3.22: Решение

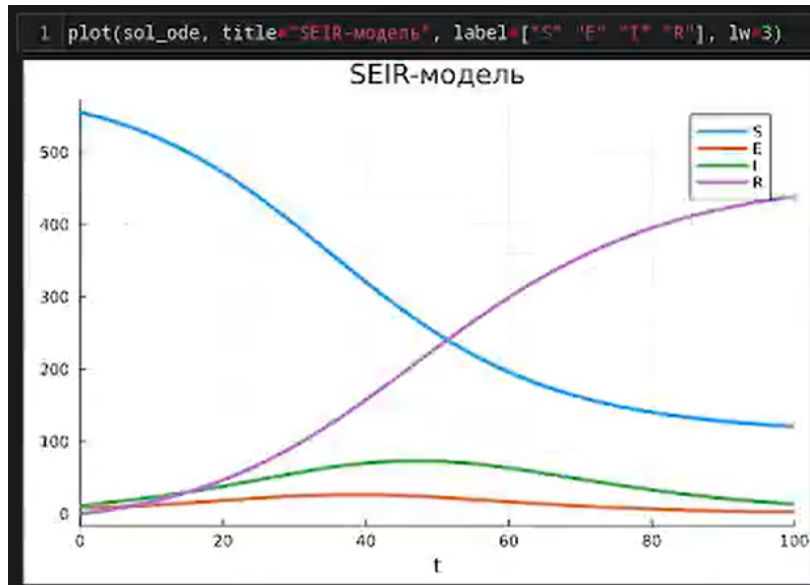


Рисунок 3.23: График

№5.

Для дискретной модели Лотки–Вольтерры:

$$\begin{cases} X_1(t+1) = aX_1(t)(1 - X_1(t)) - X_1(t)X_2(t), \\ X_2(t+1) = -cX_2(t) - dX_1(t)X_2(t). \end{cases}$$

с начальными данными $a = 2, c = 1, d = 5$ найдём точку равновесия. Получим и сравним аналитическое и численное решения. Численное решение изобразим на фазовом портрете.

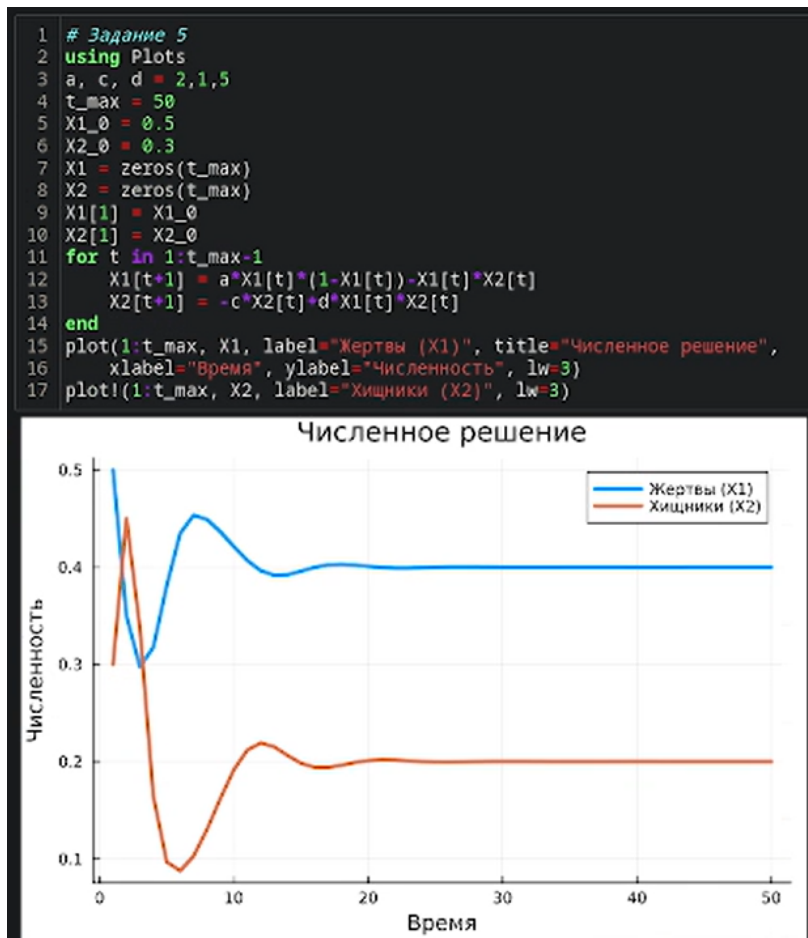


Рисунок 3.24: График

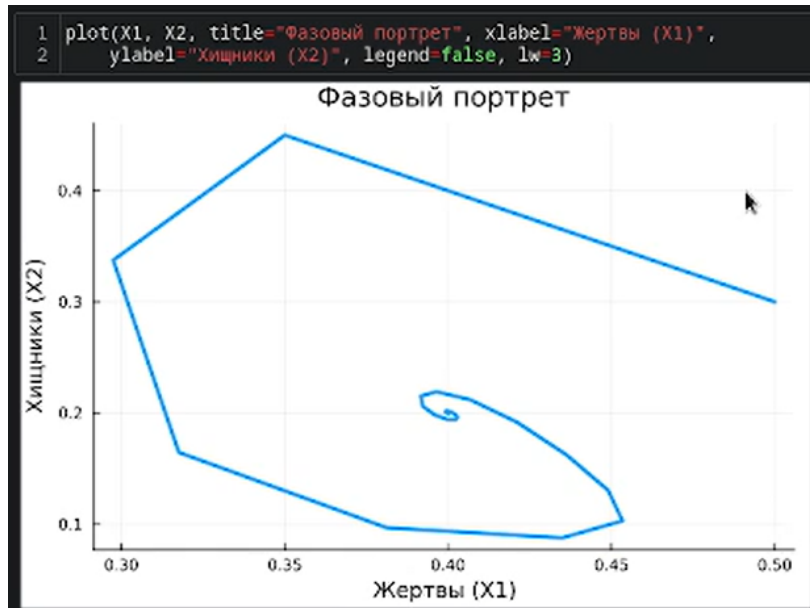


Рисунок 3.25: Фазовый портрет

№6.

Реализуем на языке Julia модель отбора на основе конкурентных отношений:

$$\begin{cases} \dot{x} = \alpha x - \beta xy, \\ \dot{y} = \alpha y - \beta xy, \end{cases}$$

Построим соответствующие графики (в том числе с анимацией) и фазовый портрет.

```

1 # Задание 6
2 function competition(u,p,t)
3     (x, y) = u
4     a, beta = p
5     dx = a*x - beta*x*y
6     dy = a*y - beta*x*y
7     return [dx, dy]
8 end
9 tspan = (0.0, 6.0)
10 u0 = [10, 5]
11 p = [1, 0.01]
12 prob = ODEProblem(competition, u0, tspan, p)
13 sol = solve(prob, Rodas5())

retcode: Success
Interpolation: specialized 4th (Rodas6P = 5th) order "free" stiffness-aware interpolation
t: 21-element Vector{Float64}:
 0.0
 0.10156702565895448
 0.3334551644126046
 0.6226566940365708
 0.9701613200703951
 1.3708788049448486
 1.8287849574795136

```

Рисунок 3.26: Решение

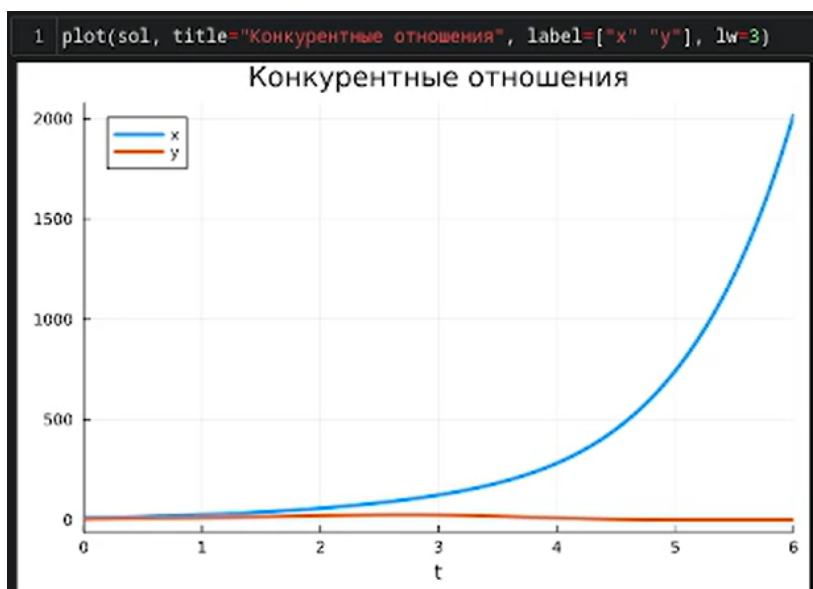


Рисунок 3.27: График

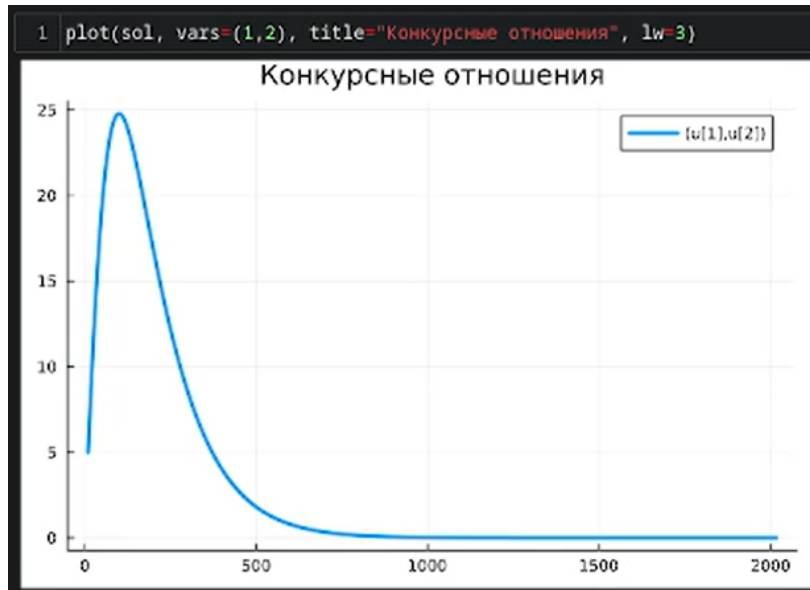


Рисунок 3.28: Фазовый портрет

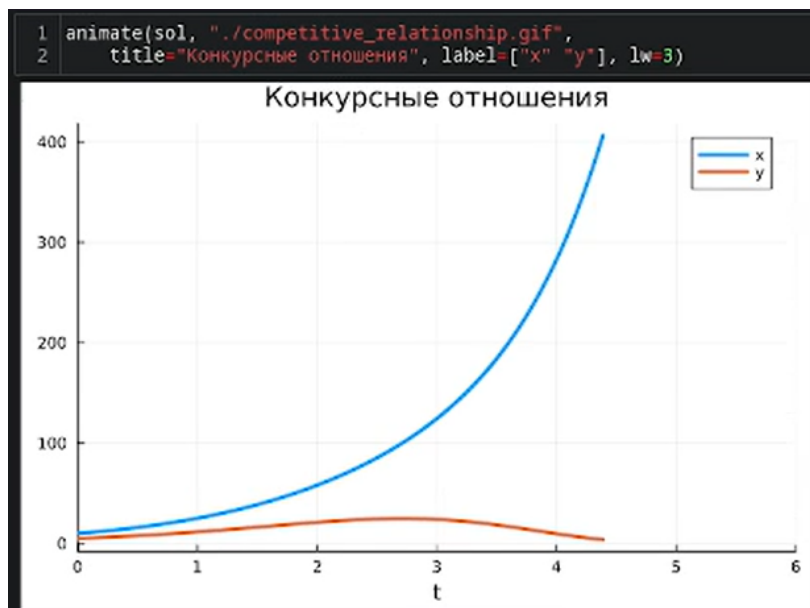


Рисунок 3.29: Анимация

№7.

Реализуем на языке Julia модель консервативного гармонического осциллятора:

$$\ddot{x} + \omega_0^2 = 0, x(t_0) = x_0, \dot{x}(t_0) = y_0,$$

где ω_0 – циклическая частота. Построим соответствующие графики (в том числе с анимацией) и фазовый портрет.

```

1  # Задание 7
2  function harmonic(u,p,t)
3      x, y = u
4      y, w = p
5      dx = y
6      dy = -2*y*y - w^2*x
7      return [dx, dy]
8  end
9  tspan = (0.0, 2*pi)
10 u0 = [-0.5, 0]
11 p = [0, 5]
12 prob = ODEProblem(harmonic, u0, tspan, p)
13 sol = solve(prob, Tsit5())

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 47-element Vector{Float64}:
 0.0
 7.984031936127744e-5
 0.0008782435129740517
 0.008862275449101793
 0.03395555370560426
 0.07658774742256073
 0.13245479683802777
 0.20450511772388788
 0.2910194975533972
 0.3860222728378222
 0.48580807986129465

```

Рисунок 3.30: Решение

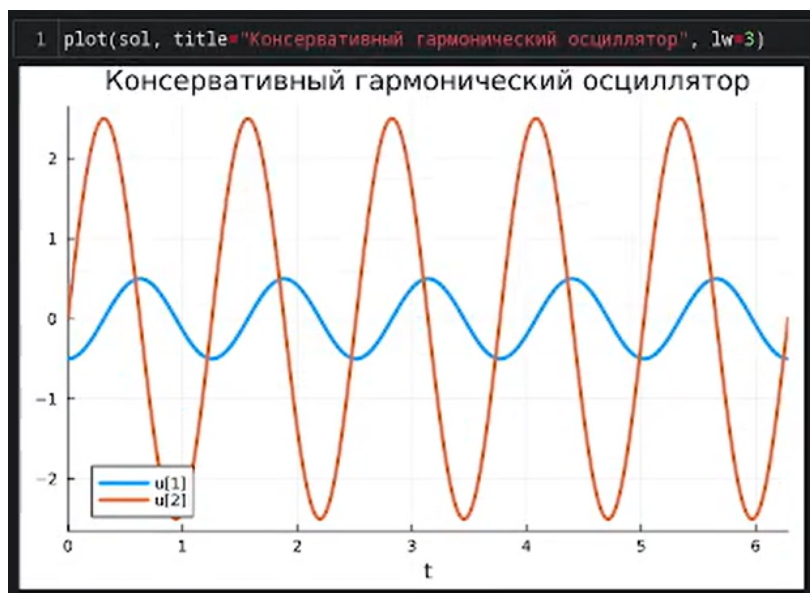


Рисунок 3.31: График

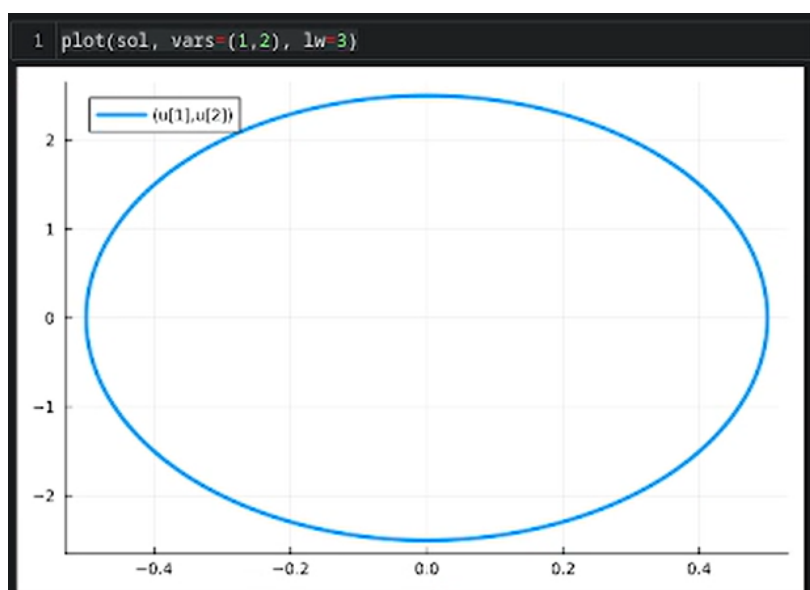


Рисунок 3.32: Фазовый портрет

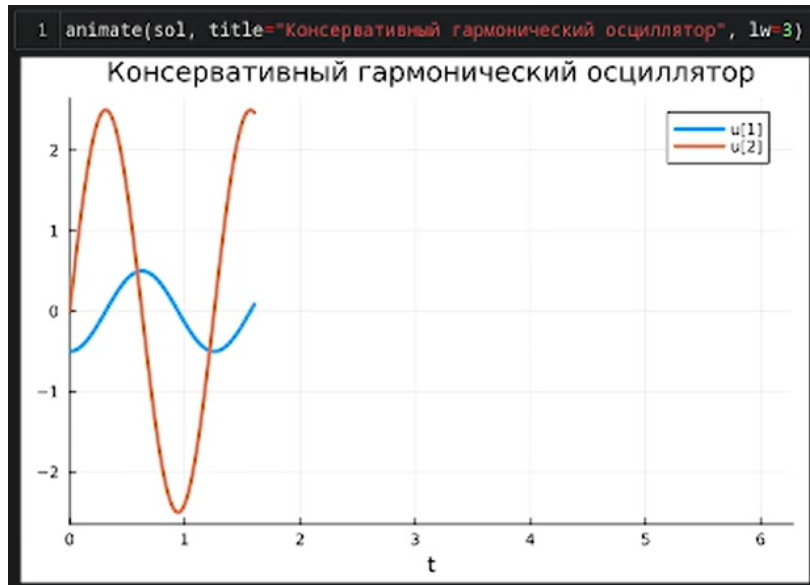


Рисунок 3.33: Анимация

№8.

Реализуем на языке Julia модель свободных колебаний гармонического осциллятора:

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 = 0, x(t_0) = x_0, \dot{x}(t_0) = y_0.$$

где ω_0 – циклическая частота, γ – параметр, характеризующий потери энергии. Построим соответствующие графики (в том числе с анимацией) и фазовый портрет.

```

1 # Задание 8
2 function harmonic2(du, u , p, t)
3     γ, ω0 = p
4     du[1] = u[2]
5     du[2] = -2*γ*u[2]-ω0^2*u[1]
6 end
7 u0 = [1.0, 0.0]
8 γ = 0.1
9 ω0 = 1.0
10 p = (γ, ω0)
11 tspan = (0.0, 70.0)
12 prob = ODEProblem(harmonic2, u0, tspan, p)
13 sol = solve(prob, Tsit5())

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 93-element Vector{Float64}:
 0.0
 0.0009999999999999992
 0.010989010989010992
 0.0810579362289679
 0.24521737536858496

```

Рисунок 3.34: Решение

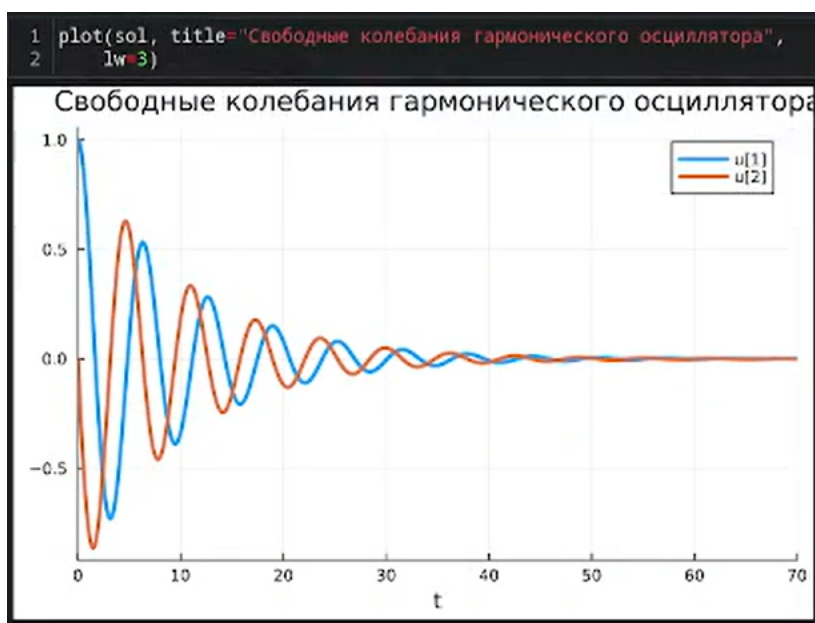


Рисунок 3.35: График

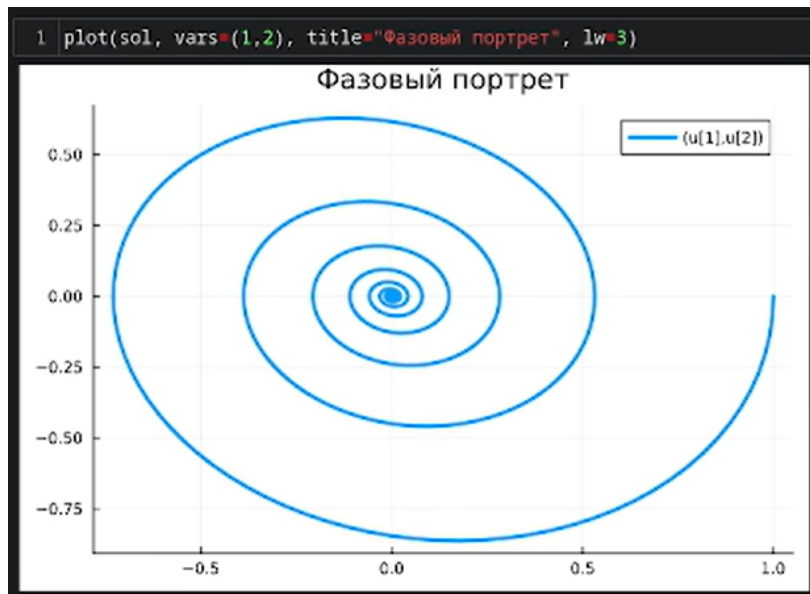


Рисунок 3.36: Фазовый портрет

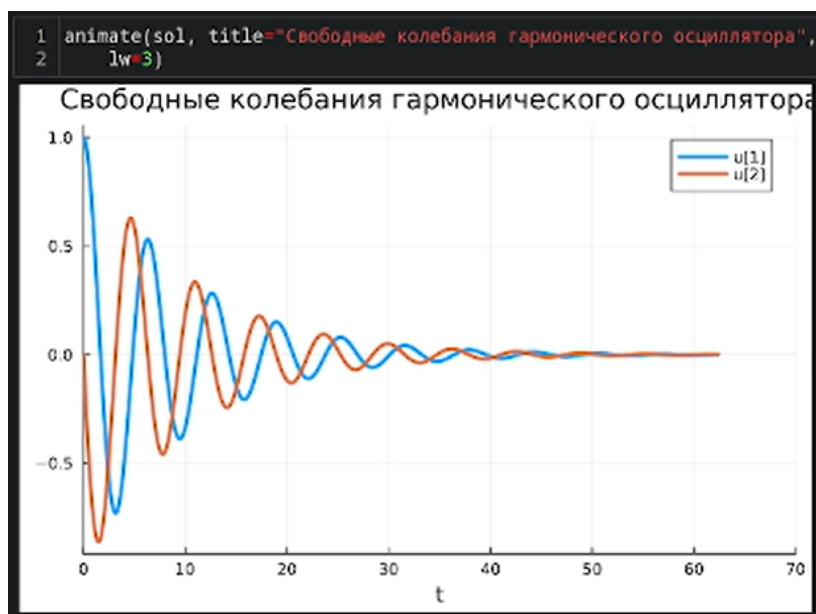


Рисунок 3.37: Анимация

4 Выводы

Освоены специализированные пакеты для решения задач в непрерывном и дискретном времени.

Список литературы

- [1] Julia Language Documentation. *Julia 1.5 Documentation*. 2020. URL: <https://docs.julialang.org/en/v1/>.